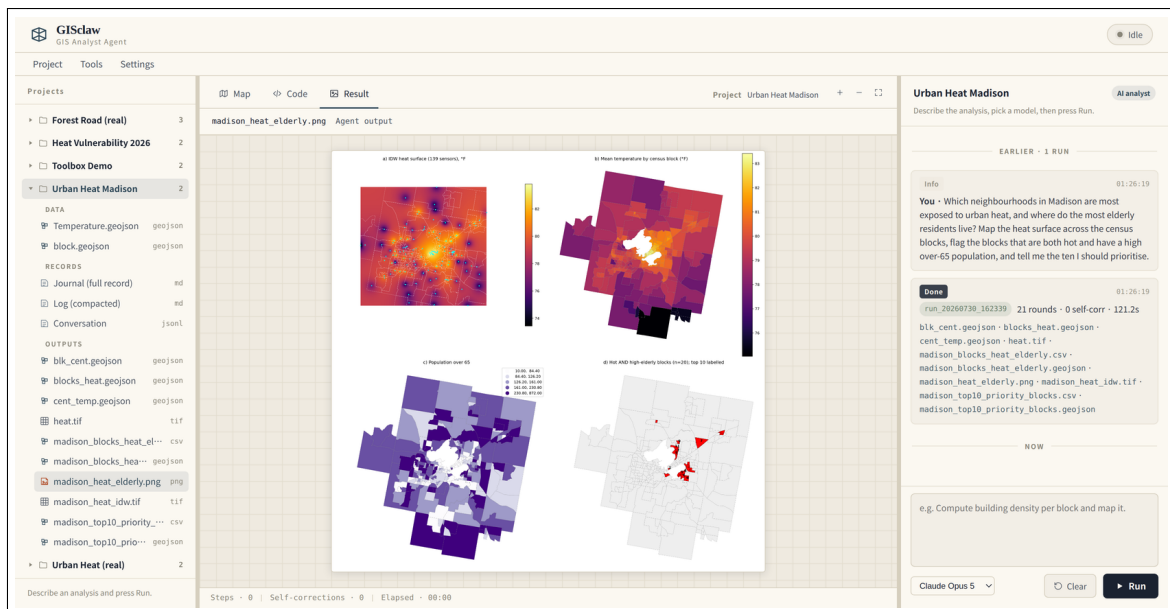


GISclaw

User Manual

Describe the spatial analysis you need. Get the finished work.



GISclaw is an automation tool for geospatial analysis. You state the question in ordinary language and it plans the workflow, runs it against your own data, and returns the layers, maps, tables, and a written account of how it got there. This manual covers installation, day-to-day use, the record it keeps, and the mechanisms that make a months-long project stay legible.

Contents

1	What GISclaw does	2
1.1	Typical requests	2
2	Installation	2
2.1	Requirements	2
2.2	Steps	2
3	Daily use	3
3.1	Creating a project and attaching data	3
3.2	Writing a good request	3
3.3	Watching a run	4
4	How a run is governed	5
5	What every project keeps	5
5.1	The compacted log	6
6	Different requests, different responses	6
7	Standing preferences	7
8	Skills	7
9	The operations toolbox	7
10	Where your data goes	9
11	Troubleshooting	9
12	Relationship to the research paper	9
13	Licence and attribution	9

1 What GISclaw does

Spatial analysis is largely a sequence of well-understood operations assembled in the right order: reproject, interpolate, join, summarise, classify, map. The assembly consumes the hours, and it is where mistakes hide.

GISclaw accepts the request in the form a domain scientist would actually say it. The four-panel figure on the cover was produced from this single sentence:

Which neighbourhoods in Madison are most exposed to urban heat, and where do the most elderly residents live? Map the heat surface across the census blocks, flag the blocks that are both hot and have a high over-65 population, and tell me the ten I should prioritise.

From that, GISclaw inspected both layers, found they used different coordinate reference systems, reprojected, built an IDW heat surface from 139 sensor readings, computed zonal means for 269 census blocks, normalised and combined heat with over-65 density, ranked the result, drew the figure, and wrote ten files. Twenty-one steps, 121 seconds.

1.1 Typical requests

You say	It produces
Where would flooding above 2 m cut off access to the hospital?	inundation extent, affected road segments, isolated catchments
How much protected forest falls within 500 m of the planned road?	buffers, intersections, affected area by protection class, an impact map
Which districts are underserved by fire stations?	service-area buffers, coverage gaps, population inside each gap
Compare forest cover between these two dates and quantify the loss.	change raster, gain/loss/net figures, a diverging-scheme map

2 Installation

2.1 Requirements

Docker and Docker Compose, roughly 3 GB of disk for the image, and an API key for one model provider. No GIS software needs to be installed: the container carries the full open-source geospatial stack.

2.2 Steps

1. `git clone <repository>` and `cd GISclaw`
2. `docker compose up -d`
3. Open `http://localhost:8765`
4. **Settings** → **API keys...**, paste a key, press **Test**

Where keys live. Keys are stored server-side under `./projects/.gisclaw/settings.json` with file mode 600. They never enter the image, never reach the browser, and are returned to the interface only as a mask such as `sk-abc...wxyz`. Environment variables in a `.env` file work as a fallback for CI or shared hosts.

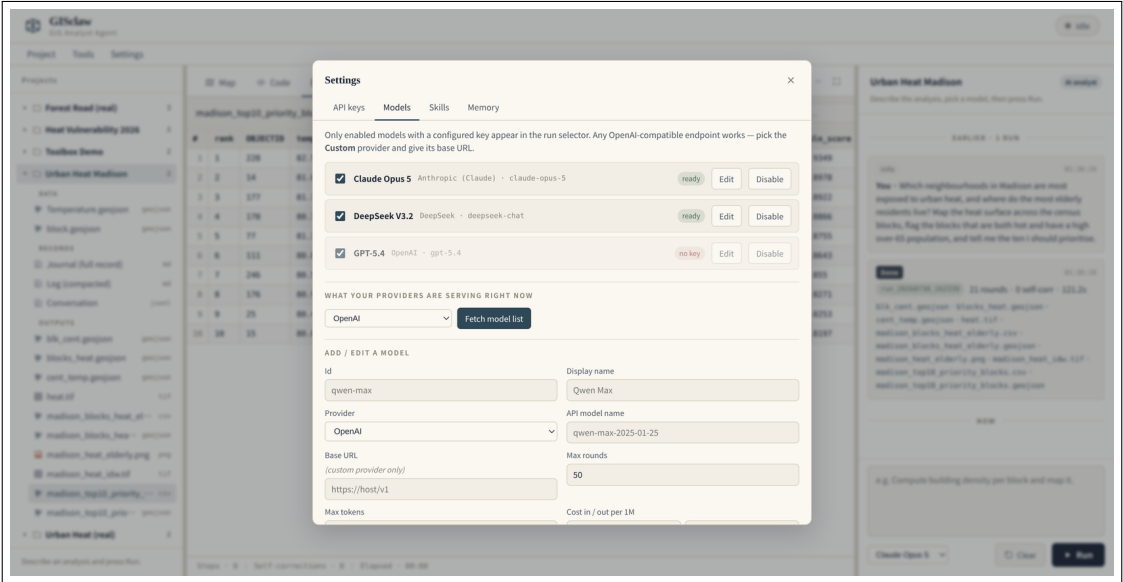


Figure 1: Model registry. The list is fetched live from each provider, so newly released models appear without an update to this software; only models with a working key are offered in the run selector.

3 Daily use

3.1 Creating a project and attaching data

A project is a working folder. Create one with **Project** → **New project...**, then attach data with **Project** → **Add data...**, which opens a server-side file browser rooted at the mounted workspace. You can also right-click a project folder in the tree.

3.2 Writing a good request

State the goal, name any constraint that matters, and say what you want back. The agent reads the schema itself, so column names rarely need to be supplied.

Weak	Analyse the heat.
Workable	Interpolate the temperature points and summarise by block.
Strong	Interpolate the TemperatureF points into a continuous surface, compute the zonal mean for each census block, save blocks_meantemp.geojson and a choropleth PNG to pred_results/, and report the five hottest blocks and how many blocks have no value.

Asking for a number you can check — the five hottest blocks, the count with no data — is the cheapest way to catch a run that finished without being correct.

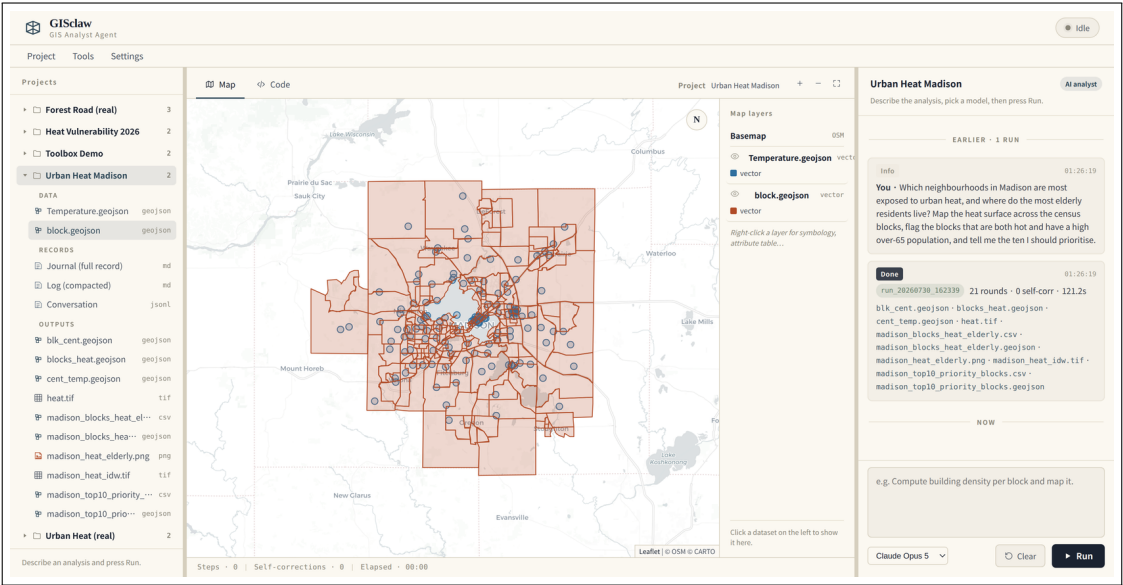


Figure 2: Layers rendered on the map, project tree on the left, conversation on the right. Right-click any layer for symbology, an attribute table, or zoom.

3.3 Watching a run

Thought, Action, and Observation stream into the conversation as they happen. Generated code appears in the **Code** tab, figures in **Result**, tables and text in **Table**, and vector or raster outputs render on the map automatically.

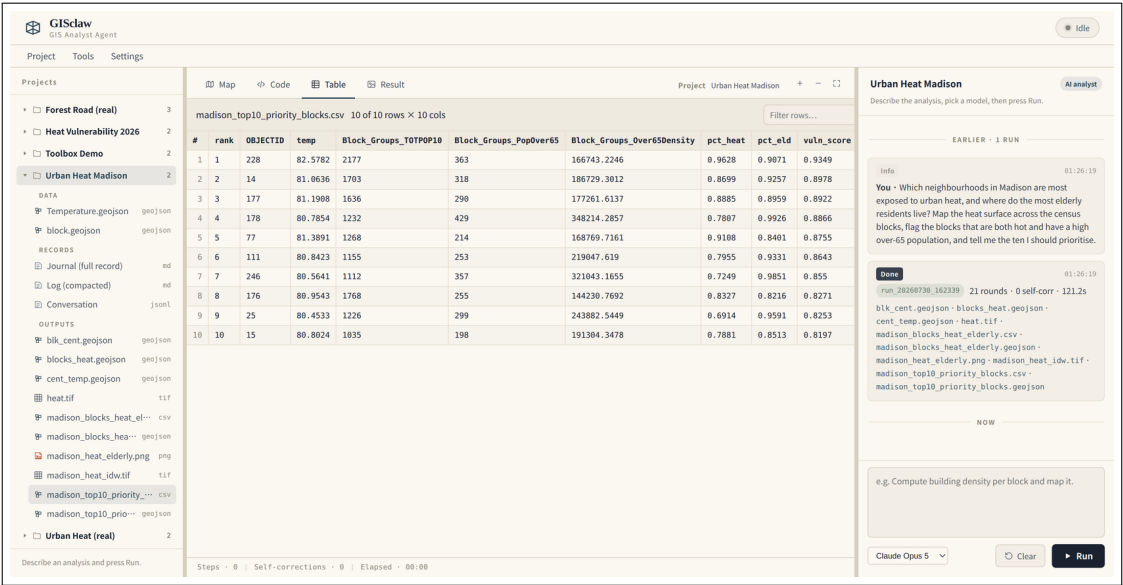


Figure 3: The request, the run summary with its outputs, and a produced table opened in the viewer.

4 How a run is governed

GISclaw drives a ReAct loop over a persistent Python sandbox: look at the data, reason about the next step, write a few lines, read the result, continue.

The operating discipline comes from roughly 1,800 controlled runs of this agent on real multi-step GIS tasks, and is applied to every analysis.

1. **Plan the whole path first.** The dominant failure is correct code for the wrong plan.
2. **Read the schema from the data.** Column names are never assumed.
3. **Treat CRS as a first-class concern.** Mixed projections are the norm; distance and area work happens in a projected system, and the null rate is printed after every join.
4. **Prefer the deterministic operations** over hand-written equivalents.
5. **Never fill in missing values quietly.** Units with no measurement stay null. Where filling is unavoidable, the reason, the count, and a flag column are required, and it must appear in the summary.
6. **Diagnose rather than retry.** The real error is at the end of a traceback.
7. **Verify before finishing.** Ranges, counts, and whether the mapped field varies at all.

Why rule 5 exists. A run that fills 170 of 269 units with the mean of the other 99 produces a complete-looking map in which two thirds of the pattern is manufactured. It looks like success in every automated check. Coverage of 99 units, stated plainly, is worth more.

5 What every project keeps

projects/<your project>/	
├─ data/	what you attached
├─ outputs/	what came out
├─ JOURNAL.md	full record: every run, every round
├─ LOG.md	compacted digest, one entry per run
├─ chat.jsonl	the conversation, rebuilt in the UI
└─ runs/run_<timestamp>/	
├─ code.py	exactly what was executed
├─ trace.jsonl	every Thought / Action / Observation
└─ pred_results/	that run's outputs, never overwritten

Refresh the browser, restart the container, or return in three months: the conversation is restored from `chat.jsonl`. Click any past run to replay its reasoning and load the code it executed.

Which copy to trust. `outputs/` is a convenience copy that a later run may overwrite with the same filename. `runs/.../pred_results/` is never modified, so a specific result is always reproducible from there.

5.1 The compacted log

Long projects accumulate transcripts nobody rereads. After each run GISclaw spends one model call compressing the trace into five fields — Result, Method, Numbers, Caveats, Carries forward — and appends it to LOG.md. That digest is what feeds the next run's context.



Figure 4: The compacted log. Each entry is written by the model from that run's full trace.

6 Different requests, different responses

GISclaw reads what kind of message it received before acting on it, which makes it usable through the whole arc of a piece of work rather than only at the moment you already know what to run.

- **A request for analysis** starts a run.
- **A question about the project** — what was done, what a layer holds, why a value looks odd — is answered from the record in a single call.
- **A proposal you want to think through** gets discussed. Sketch an approach, ask what it would do with the data you have, weigh two methods against each other, and settle the plan before any computation happens.
- **Ordinary conversation** stays ordinary.
- **Anything outside its scope** is declined with a note about what it does cover.

Discussing a method costs one short call; running it can cost several minutes and a dollar. Being able to agree on the approach first is often the cheaper path to the right answer.

7 Standing preferences

A global `MEMORY.md` holds conventions that apply to everything: colour ramps, the classification scheme you standardise on, the projected CRS for your region, what every deliverable map must carry. Write it once under **Settings** → **Memory...**, or hover any message and press **Remember**. It is injected into every run.

Keep it to rules. Text inside HTML comments is stripped before injection, so notes to yourself cost nothing.

8 Skills

A skill is a directory bundle that encodes how a class of analysis should be done:

```
<skill>/
├─ SKILL.md          YAML frontmatter and a short router
├─ manifest.yaml     optional declarative loading plan
└─ references/*.md   the depth, opened only when a step calls for it
```

Loading is progressive, which keeps a large library affordable: a one-line description stays in context permanently (around 80 tokens), the router loads when the skill becomes relevant, and reference files open one at a time.

Level	Content	When it loads
Catalogue	name and one-line description	always
Router	SKILL.md body and file listing	when the skill is opened or matched
Depth	one reference file	when a step calls for it

Two skills ship with the application. `gis-analysis-discipline` carries the rules in section 4 and is always on. `gis-workflow-recipes` holds step-by-step templates for interpolation with zonal summary, multi-criteria suitability overlay, proximity and impact assessment, and two-date change detection.

Bundles import as a `.zip` or a folder, edit in the application, and export for sharing. Authors should declare keywords: in the frontmatter, which is what server-side matching uses to pre-load the right skill.

9 The operations toolbox

Twenty-eight deterministic geoprocessing operations ship with the application: reproject, buffer, clip, intersection, difference, union, dissolve, spatial and attribute joins, zonal statistics, slope, aspect, hillshade, rasterize, IDW, and more.

They serve two practical purposes. They are tested and CRS-aware, so the agent reaches for them ahead of writing equivalent code by hand. And when you already know the exact operation you want, running it directly from the panel costs no API call at all, with the result rendered on the map immediately.

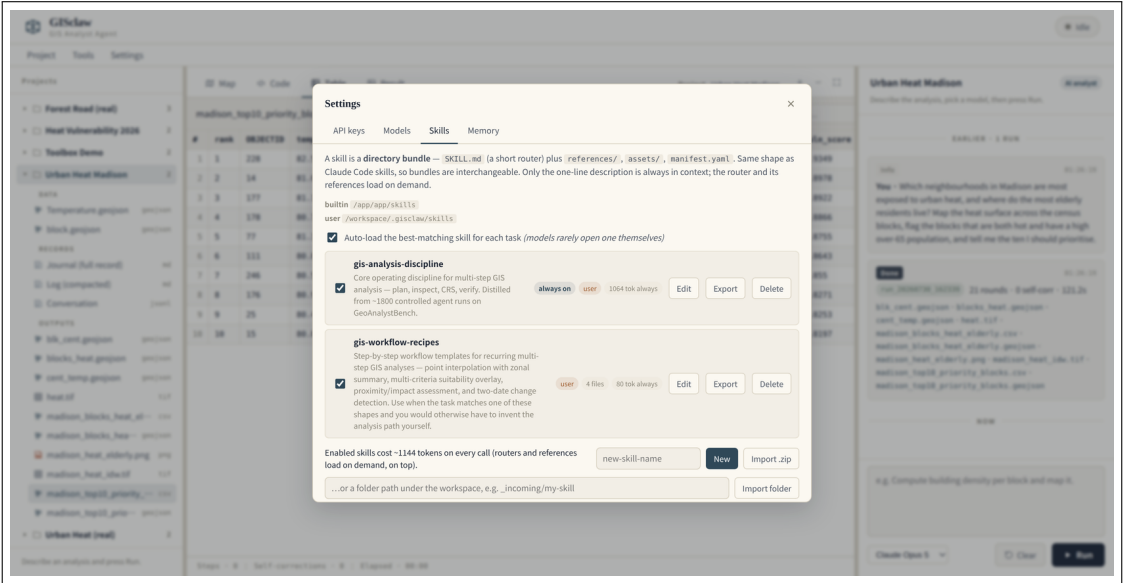


Figure 5: The skills panel, showing source, bundled file count, and the token cost each enabled skill adds.

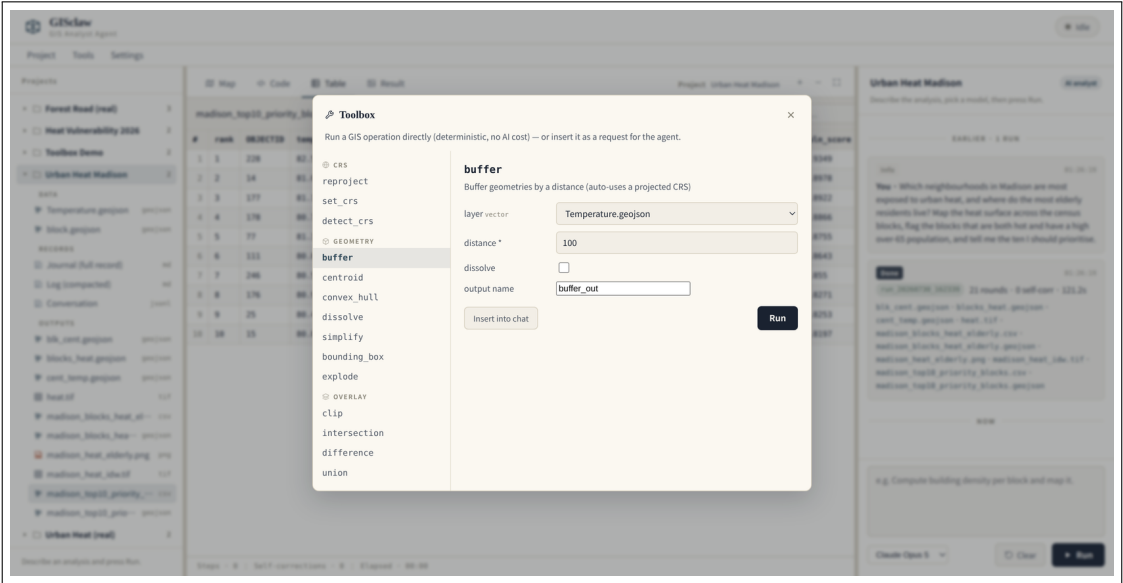


Figure 6: Each operation exposes a real parameter form. **Run** executes it deterministically; **Insert into chat** hands it to the agent as one step of a larger workflow.

10 Where your data goes

Analysis runs inside the container against your native formats — Shapefile, GeoTIFF, GeoPackage, CSV, NetCDF. Conversion to GeoJSON or PNG happens only in the browser display layer, since that is what a browser can draw. The files stay on your machine. What reaches the model provider is your prompt and the agent’s own observations of the data.

11 Troubleshooting

Symptom	Check
Model selector reads No model configured	No provider has a key. Settings → API keys, then Test .
A run fails immediately with a key error	The selected model’s provider has no key; choose a model marked ready.
Raster operations fail on the host	Run rasters inside the container; a broken host PROJ database affects rasterio.warp but not vector work.
The interface looks stale after an update	Hard-refresh the browser and confirm the console reports the expected app.js version.
Intermediate files appear in outputs/	Each geoprocess step writes to pred_results/; intermediates are copied along with final products.

Server logs: `docker logs -f gisclaw_app`. Per-run logs: `projects/<project>/runs/<run_id>/run.log`.

12 Relationship to the research paper

The system described in arXiv:2603.26845 is a benchmark harness built for controlled experiments. This desktop application was rebuilt on that agent core and has since diverged substantially: an operations toolbox usable from the interface, project persistence with journal and compacted log, skill bundles with progressive disclosure, global preference memory, and live model discovery.

The experimental results reported in the paper came from the research harness at its frozen commit. Reproduction work should use that artifact.

13 Licence and attribution

Copyright © 2026 Han Jinzhen. GISclaw is free software under the GNU Affero General Public License, version 3 or later. You may use it for any purpose, including commercially, at no cost. If you distribute a modified version, or run one as a network service that other people use, section 13 of the AGPL requires you to offer those users the complete corresponding source of your version under the same licence. Running it unmodified, or modifying it only for your own internal use, obliges you to publish nothing.

A separate commercial licence is available from the copyright holder for embedding GISclaw in a proprietary product or operating a closed-source hosted service; see

`COMMERCIAL-LICENSE.md`. The research code behind the companion paper remains MIT-licensed on the `paper-v2` branch, so that published results stay reproducible under the terms the paper cites.

The bundled example dataset comes from GeoAnalystBench (Zhang et al., 2025) under Apache-2.0 and carries its own citation requirement; see `examples/urban-heat-madison/README.md` and `THIRD_PARTY_NOTICES.md` for the full list of third-party material.